ACO shortest-path in a network visualizer

Sun HeYang F112088
Zhao shengji F116120
Dmitrii Temnov F110731

Project Description

- generates a random network that is composed by a random graph G =（V，E）and a weight function w : E -> N that maps the set of edges E to the natural numbers -> N;
- given a source vertex u and a destination vertex v, calculates the shortest path between them using the ACO algorithm;
- demonstrates visually the execution of the ACO algorithm: how ants select the path, where pheromone is placed and how it evaporates.

Graph

The bidirectional graph is represented as an adjacency matrix of weights, implemented as a nested map map<string, map<string, int>> adjMatrix. Vertices are identified by string names, edges are identified as a pair of vertices.

The graph class supports adding or removing vertices and edges as well as viewing and editing the weights of graph edges.

ACO core components

1. ACOService (`ACOService.h`, `ACOService.cpp`)

The main class implementing the ACO algorithm. It manages:

- **Graph representation**: Uses the `Graph` class to represent the weighted graph
- **Pheromone matrix**: Maintains pheromone levels for each edge in the graph
- **ACO parameters**: Configurable parameters that control algorithm behavior

2. Graph Structure (`graph.h`, `graph.cpp`)

Represents the graph as an adjacency matrix using nested maps:

map<string, map<string, int>> adjMatrix

- Vertices are identified by string names
- Edge weights are stored as integers
- Supports bidirectional edges

ACO core components

3. AcoRunner (`AcoRunner.h`, `AcoRunner.cpp`)

Wrapper class that:

- Converts the `Graph` structure to an adjacency matrix format
- Executes the ACO algorithm
- Handles multiple attempts if no path is found initially
- Calculates the total weight of the resulting path

ACO Parameters

Parameter	Value	Description
`alpha`	1.0	Pheromone importance factor - controls how much ants rely on existing pheromone trails
`beta`	2.0	Heuristic importance factor - controls how much ants prefer shorter edges
`evaporationRate`	0.4	Rate at which pheromones evaporate (40% per iteration)
`depositAmount`	100.0	Base amount of pheromone deposited by ants
`antsNumber`	N	Number of ants equals the number of vertices in the graph
`maxIterations`	100	Maximum steps an ant can take before stopping
`pheromoneIterations`	30	Number of outer iteration rounds

Algorithm flow

The `findPath(source, destination)` method of ACOService class implements the following steps:

1. Initialization

// Initialize pheromone matrix with value 1.0 for all edges
pheromoneMatrix[from][to] = 1.0;

2. Main Loop (30 iterations)

For each iteration:

a. Ant Creation

- Create `N` ants (where `N` = number of vertices)
- Each ant is represented by vector<string> where strings are the names of vertices, visited by the ant
- All ants start at the source vertex

Algorithm flow

b. Ant Movement

Each ant moves until it reaches the destination or exceeds `maxIterations`:

- **Current position**: Get the ant's current vertex
- **Next vertex selection**: Use probabilistic selection based on transition probability

Transition probability calculation:

$P(i,j) = (\tau_i^{\alpha} \times \eta_i^{\beta}) / \Sigma(\tau_i^{\alpha} \times \eta_i^{\beta})$
Where:

- τ_i = pheromone level on edge (i,j)
- η_i = 1/weight(i,j) (heuristic value - shorter edges are more attractive)
- α = pheromone importance (1.0)
- β = heuristic importance (2.0)
- The sum is over all available unvisited neighbors

Roulette wheel selection: Randomly select next vertex based on calculated probabilities

If an ant gets stuck (no available unvisited neighbors), it resets to the source vertex.

Algorithm flow

c. Pheromone Update

After all ants complete their paths the program updates pheromones on all the edges:

Deposit: Ants that successfully reached the destination deposit pheromones:

deposit = depositAmount / pathLength

This prevents the algorithm from getting stuck in local optima and allows exploration of new paths.
Shorter paths receive more pheromone per edge, making them more attractive in future iterations.

Evaporation: All edges lose pheromone:

new_pheromone = old_pheromone × (1 - evaporationRate)

3. Best Path Selection

After all the iterations are done, return the shortest path found among all successful ant paths.

Graph GUI

- Node :
  Name
  Click event
  Appearance

- Edge:
  Weight
  Appearance

Node

```h
#ifndef NODEITEM_H
#define NODEITEM_H

#include<QGraphicsEllipseItem>
#include<QGraphicsSceneMouseEvent>
#include<QBrush>
#include<QString>

class NodeItem : public QGraphicsEllipseItem
{
        public:
                NodeItem(const QString& name, const QPointF& pos, qreal radius = 20);

                QString getName() const {return m_name;}
        protected:
                void mousePressEvent(QGraphicsSceneMouseEvent* event) override;
        private:
                QString m_name;
};

#endif
```


```cpp
#include"NodeItem.h"
#include<QPen>
#include<QGraphicsTextItem>

NodeItem::NodeItem(const QString& name, const QPointF& pos, qreal radius) : m_name(name)
{
        setRect(pos.x() -radius, pos.y() -radius, radius *2, radius *2); //
  QGraphicsEllipseItem draw circle as a circle inside of a rectangle, need left upper corner
  point,height,width
        setBrush(QBrush(Qt::yellow));
        setPen(QPen(Qt::black, 2));

        //show Node name
        auto* text = new QGraphicsTextItem(name, this);
        text->setPos(pos.x()-6, pos.y()-10);
}

void NodeItem::mousePressEvent(QGraphicsSceneMouseEvent* event)
{
        setBrush(QBrush(Qt::green));
        QGraphicsEllipseItem::mousePressEvent(event);
}
```

Edge

```
#ifndef EDGEITEM_H
#define EDGEITEM_H

#include<QGraphicsLineItem>
#include<QGraphicsTextItem>

class EdgeItem : public QGraphicsLineItem
{
        public:
                EdgeItem(const QPointF& p1, const QPointF& p2, int weight);
        private:
                QGraphicsTextItem* weightText;
};

#endif
```

```
#include"EdgeItem.h"
#include<QPen>

EdgeItem::EdgeItem(const QPointF& p1, const QPointF& p2, int weight)
{
        setLine(QLineF(p1,p2));
        setPen(QPen(Qt::black, 2));

        QPointF mid = (p1 + p2) / 2;
        weightText = new QGraphicsTextItem(QString::number(weight), this);
        weightText->setPos(mid.x(), mid.y());
}
```

Graph Scene

```h
#ifndef GRAPHSCENE_H
#define GRAPHSCENE_H

#include<QGraphicsScene>
#include"graph.h"
#include"NodeItem.h"
#include"EdgeItem.h"

class GraphScene : public QGraphicsScene
{
        public:
                GraphScene(QObject* parent = nullptr);

                void drawGraph(Graph& graph);

        private:
                QMap<QString, NodeItem*> nodeItems;
};

#endif
```

```cpp
#include"GraphScene.h"
#include<QRandomGenerator>

GraphScene::GraphScene(QObject* parent) : QGraphicsScene(parent)
{
}

void GraphScene::drawGraph(Graph& graph)
{
        clear();
        nodeItems.clear();

        auto vertices = graph.getVertices();

        for (const auto& v: vertices)
        {
                QPoint pos(
                                QRandomGenerator::global()->bounded(100,600),
                                QRandomGenerator::global()->bounded(100,400)
                );

        NodeItem* node = new NodeItem(QString::fromStdString(v), pos);
        addItem(node);

        nodeItems[v.c_str()] = node;
        }

        for (const auto& v1 : vertices)
        {
                for (const auto& v2 : vertices)
                {
                        if(graph.edgeExist(v1,v2))
                        {
                                QPointF p1 = nodeItems[v1.c_str()]->rect().center();
                                QPointF p2 = nodeItems[v2.c_str()]->rect().center();

                                int w = graph.getWeight(v1, v2);
                                addItem(new EdgeItem(p1, p2, w));
                        }
                }
        }
}
```

Main Window

```h
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include<QMainWindow>
#include<QGraphicsView>
#include<QSpinBox>
#include<QPushButton>
#include<QHBoxLayout>
#include<QVBoxLayout>
#include"GraphScene.h"
#include"graph.h"

class MainWindow : public QMainWindow
{
        Q_OBJECT

        public:
                MainWindow(QWidget *parent = nullptr);
        private slots:
                void choose8();
                void choose16();
                void regenerate();
        private:
                GraphScene* scene;
                QGraphicsView* view;

                int nodeCount = 8;
                QSpinBox* minWeightBox;
                QSpinBox* maxWeightBox;

                Graph generateRandomGraph();
};

#endif
```

```cpp
#include"MainWindow.h"
#include<QRandomGenerator>
#include<QLabel>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
        scene = new GraphScene(this);
        view = new QGraphicsView(scene);
        view->setRenderHint(QPainter::Antialiasing);

        QPushButton* btn8 = new QPushButton("8 Nodes");
        QPushButton* btn16 = new QPushButton("16 Nodes");
        QPushButton* regenBtn = new QPushButton("Regenerate");

        minWeightBox = new QSpinBox;
        maxWeightBox = new QSpinBox;

        minWeightBox->setRange(1, 100);
        maxWeightBox->setRange(1, 100);
        minWeightBox->setValue(1);
        maxWeightBox->setValue(10);

        QWidget* panel = new QWidget;
        QHBoxLayout* top = new QHBoxLayout;

        top->addWidget(btn8);
        top->addWidget(btn16);
        top->addWidget(new QLabel("Weight range: "));
        top->addWidget(minWeightBox);
        top->addWidget(new QLabel(" - " ));
        top->addWidget(maxWeightBox);
        top->addWidget(regenBtn);

        QVBoxLayout* main = new QVBoxLayout;
        main->addLayout(top);
        main->addWidget(view);

        panel->setLayout(main);
        setCentralWidget(panel);

        connect(btn8, &QPushButton::clicked, this, &MainWindow::choose8);
        connect(btn16, &QPushButton::clicked, this, &MainWindow::choose16);
        connect(regenBtn, &QPushButton::clicked, this, &MainWindow::regenerate);

        regenerate();
}

void MainWindow::choose8()
{
        nodeCount = 8;
        regenerate();
}

void MainWindow::choose16()
{
        nodeCount = 16;
        regenerate();
}
```

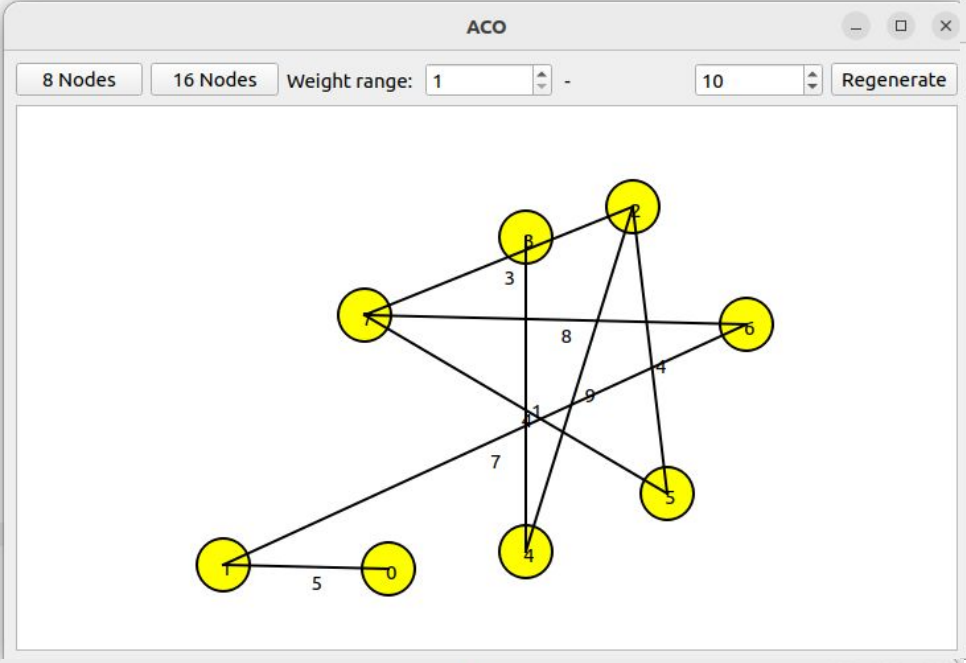
Main Window

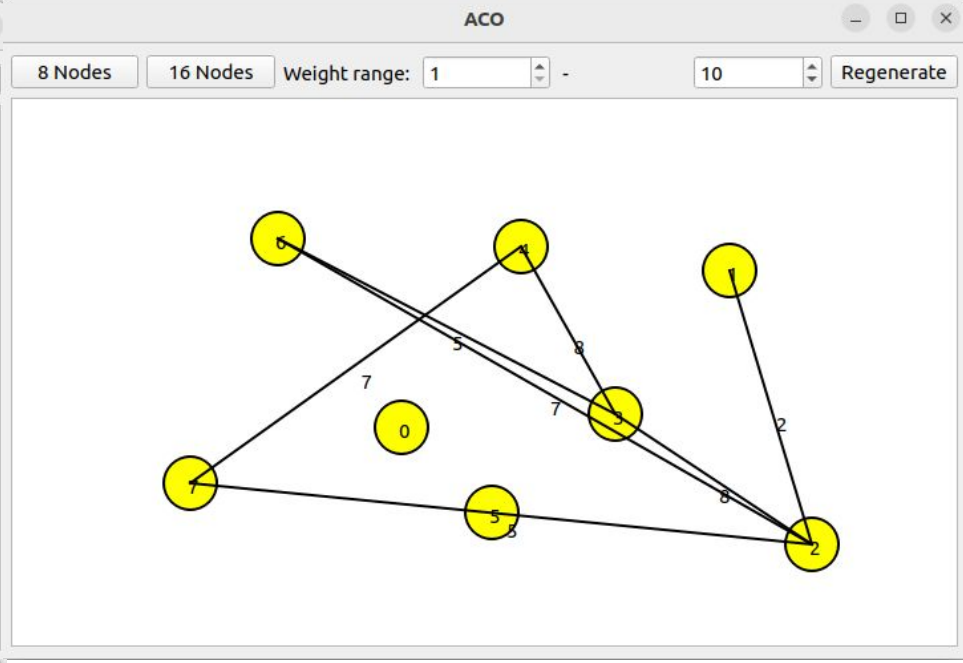
```
59
60 void MainWindow::regenerate()
61 {
62         Graph g = generateRandomGraph();
63         scene->drawGraph(g);
64 }
65
66 Graph MainWindow::generateRandomGraph()
67 {
68         Graph g;
69
70         for(int i = 0; i < nodeCount; i++) g.addVertex(QString::number(i).toStdString());
71
72         int minW = minWeightBox->value();
73         int maxW = maxWeightBox->value();
74
75         int maxEdges = nodeCount * (nodeCount - 1)/2;
76         int edgeCount = QRandomGenerator::global()->bounded(maxEdges/3, maxEdges*2/3);
77
78         for (int i = 0; i < edgeCount; i++)
79         {
80                 int u = QRandomGenerator::global()->bounded(nodeCount);
81                 int v = QRandomGenerator::global()->bounded(nodeCount);
82
83                 if(u == v) continue;
84
85                 if(g.edgeExist(QString::number(u).toStdString(), QString::number(v).toStdString()) || (g.edgeExist(QString::number(v).toStdString(), QString::number(u).toStdString())))
86                 {
87                         continue;
88                 }
89
90                 int weight = QRandomGenerator::global()->bounded(minW, maxW + 1); //bounded is [a, b) , so we need maxW+1
91
92                 g.addEdge(QString::number(u).toStdString(), QString::number(v).toStdString(), weight);
93
94         }
95
96         return g;
97 }
```

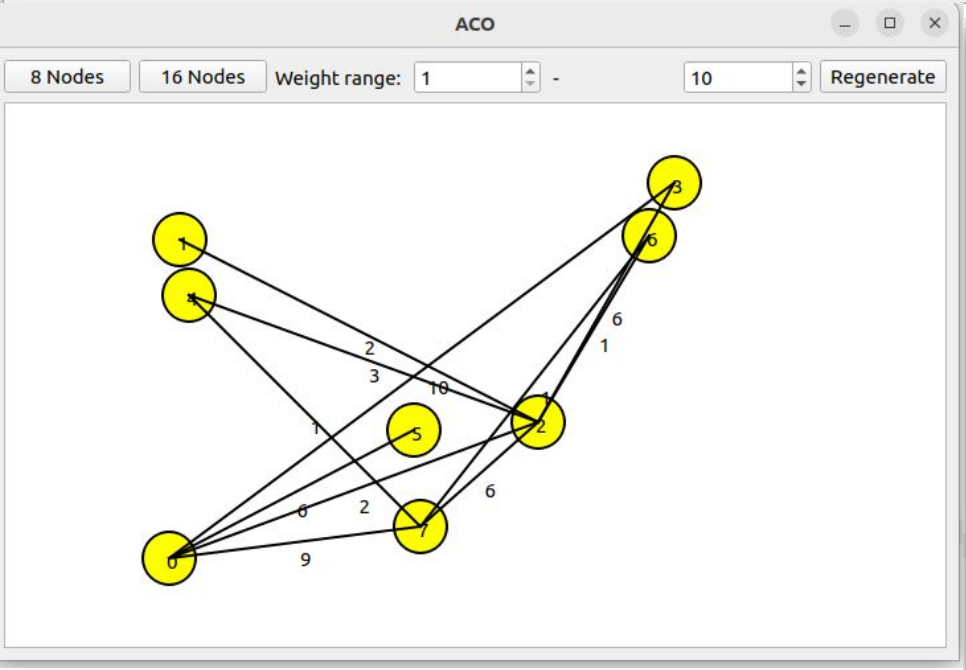
Other

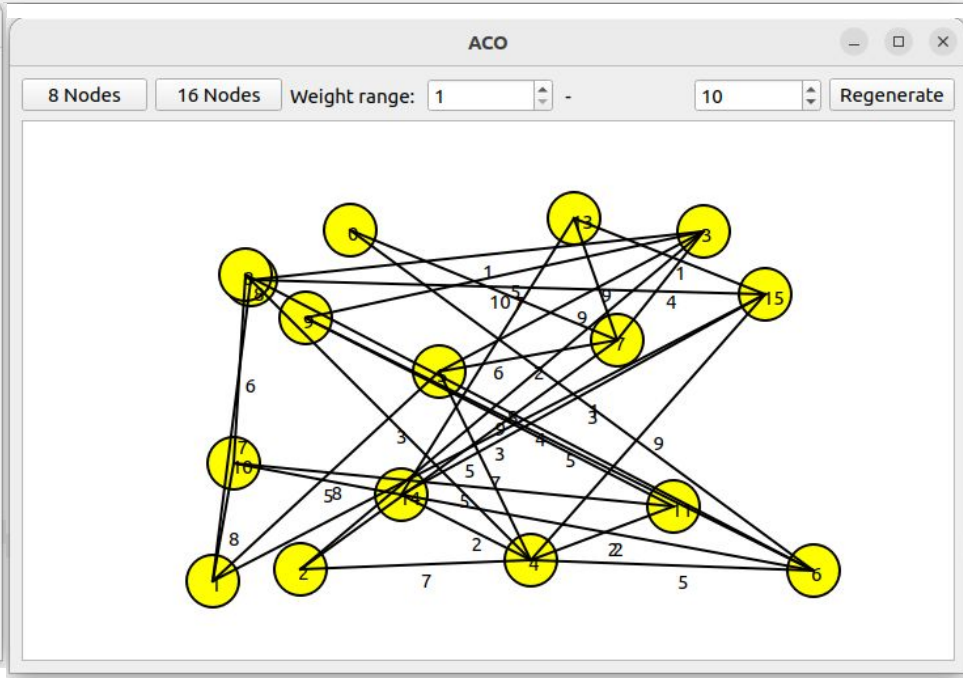
```cpp
int main(int argc, char *argv[])
{
        testGraphClass();
        testACOServiceClass();

        QApplication a(argc, argv);
        MainWindow w;
        w.show();
        return a.exec();
}
```

```
cmake_minimum_required(VERSION 3.16)
project(ACO)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_AUTOMOC ON)

find_package(Qt6 REQUIRED COMPONENTS Widgets)

set(SOURCES
        main.cpp
        MainWindow.cpp
        GraphScene.cpp
        graph.cpp
        NodeItem.cpp
        EdgeItem.cpp
)

set(HEADERS
        MainWindow.h
        GraphScene.h
        graph.h
        NodeItem.h
        EdgeItem.h
)

qt_add_executable(ACO
        ${SOURCES}
        ${HEADERS}
)

target_link_libraries(ACO
        PRIVATE Qt6::Widgets
)
```


ACO
8 Nodes
16 Nodes
Weight range:
1
-
10
Regenerate


ACO
8 Nodes
16 Nodes
Weight range:
1
-
10
Regenerate


ACO
8 Nodes
16 Nodes
Weight range:
1
-
10
Regenerate


ACO
8 Nodes
16 Nodes
Weight range:
1
-
10
Regenerate

AcoController

```
AcoController::AcoController(SelectionController* selector, QWidget* parent)
    : QObject(parent), m_selector(selector), m_parentWidget(parent)
{
    // Create widgets
    m_runBtn = new QPushButton("Run ACO", parent);
    m_resetBtn = new QPushButton("Reset", parent);
    m_infoLabel = new QLabel(parent);
    // stop use until selection two node
    m_runBtn->setEnabled(false);
    m_infoLabel->setText("Selected: (none)");

    connect(m_runBtn, &QPushButton::clicked, this, &AcoController::onRunClicked);
    connect(m_resetBtn, &QPushButton::clicked, this, &AcoController::onResetClicked);
}

void AcoController::addToLayout(QHBoxLayout* layout)
{
    layout->addWidget(m_runBtn);
    layout->addWidget(m_resetBtn);
    layout->addWidget(m_infoLabel);
}

QString AcoController::selectionText() const
{
    if (!m_selector) return "Selected: (none)";
    if (!m_selector->hasSource())
        return "Selected: click source";
    if (!m_selector->hasDestination())
        return "Selected: " + m_selector->source() + " -> (click destination)";

    return "Selected: " + m_selector->source() + " -> " + m_selector->destination();
}
```

```cpp
void AcoController::sync()
{
    if (!m_selector) return;
    m_runBtn->setEnabled(m_selector->ready());// Run only when ready
    m_infoLabel->setText(selectionText());// Update small label
}

void AcoController::onResetClicked()
{
    if (m_selector)
    {
        m_selector->clear();
    }
        sync();

}

void AcoController::onRunClicked()
{
    if (!m_selector) return;
    if (!m_selector->ready())
    {
        QMessageBox::information(m_parentWidget, "ACO", "Please select source and destination first.");
        sync();
        return;
    }

    emit runRequested(m_selector->source(), m_selector->destination());


    sync();
}
```

SelectionController

```cpp
#include"SelectionController.h"
#include"GraphScene.h"
#include <QColor>

SelectionController::SelectionController(GraphScene* scene): m_scene(scene)
{

}

void SelectionController::setScene(GraphScene* scene)
{
    m_scene = scene;
    clear();

}

void SelectionController::clear()
{
    m_source.clear();
    m_dest.clear();
    applyColors();
}
```

```cpp
void SelectionController::handleNodeClicked(const QString& nodeId)
{
    if(m_source.isEmpty())
    {
        m_source = nodeId;// dangzuo
        m_dest.clear();
        applyColors();
        return;

    }
      if (m_dest.isEmpty())
    {
        if (nodeId == m_source) return; // same no
        m_dest = nodeId;
        applyColors();
        return;
    }

      m_source= nodeId;//have both 2 node reset
      m_dest.clear();
      applyColors();
}
void SelectionController::applyColors()
{
  if (!m_scene) return;
  m_scene->resetStyles();
  if (!m_source.isEmpty())//set color
  m_scene->setNodeColor(m_source, QColor(Qt::blue));

  if (!m_dest.isEmpty())
  m_scene->setNodeColor(m_dest, QColor(Qt::red));
```

ACO Runner

```
AcoResult AcoRunner::run(Graph& graph,const std::string& source,const std::string& destination)

{
 AcoResult result;
 std::map<std::string, std::map<std::string, int>> adjacency;

for (const auto& u : graph.getVertices())
 {
for (const auto& v : graph.getVertices())
    {
if (graph.edgeExist(u, v))
  {
    adjacency[u][v] = graph.getWeight(u, v);
    adjacency[v][u] = graph.getWeight(u, v);
}
 }
}
  const int maxAttempts = 30;
for (int attempt = 0; attempt < maxAttempts; ++attempt)
{
    ACOService aco(adjacency);
    result.path = aco.findPath(source, destination);

  if (!result.path.empty())
      break;
}
  result.totalWeight = calculateTotalWeight(graph, result.path);
  return result;
}
```

```cpp
int AcoRunner::calculateTotalWeight(Graph& graph,const std::vector<std::string>& path)
{
    int sum = 0;
    if (path.size() < 2)
    return 0;
   for (size_t i = 0; i + 1 < path.size(); ++i)
     {
     const std::string& u = path[i];
     const std::string& v = path[i + 1];

    if (graph.edgeExist(u, v))
     sum += graph.getWeight(u, v);
     else if (graph.edgeExist(v, u))
     sum += graph.getWeight(v, u);

      }
return sum;
}
```

graphScene

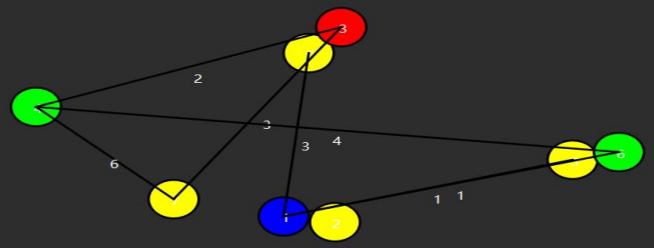
```
void GraphScene::setNodeClickHandler(const std::function<void(const QString&)>& handler)
{
    m_nodeClickHandler = handler;//The click logic is saved to the GraphScene member variable.
}//zhao

void GraphScene::resetStyles()
{
    for (auto it = nodeItems.begin(); it != nodeItems.end(); ++it)
    {
        if (it.value()) it.value()->setColor(Qt::yellow);// if empty pass or change to yellow
    }
}//zhao

void GraphScene::setNodeColor(const QString& nodeId, const QColor& color)
{
    if (!nodeItems.contains(nodeId))
    return;                              //check if ondeitems have node id,OR
    nodeItems[nodeId]->setColor(color);
}//zhao

void GraphScene::highlightPath(const std::vector<std::string>& path)
{
    resetStyles();
      for (const auto& id : path)
    {
        QString qid = QString::fromStdString(id);
        setNodeColor(qid, Qt::green);
    }
}//zhao
```

mainwindow

```
scene->setNodeClickHandler([this](const QString& nodeId)
{
 this->onNodeClicked(nodeId);
});//zhao set a return when click node weill return back mw

connect(aco, &AcoController::runRequested,this, [this](const QString& src, const QString& dst)
{
 AcoRunner runner;
 AcoResult r = runner.run(
 currentGraph,
 src.toStdString(),
 dst.toStdString()
 );
 if (r.path.empty())
 {
     scene->setNodeColor(src, Qt::blue);//even no pat color it
     scene->setNodeColor(dst, Qt::red);
     resultLabel->setText("Total path weight: No path found (graph direction/connectivity issue).");
     return;
 }
 scene->highlightPath(r.path);
 scene->setNodeColor(src, Qt::blue);
 scene->setNodeColor(dst, Qt::red);//make sure no be cover
 resultLabel->setText(QString("Total path weight: %1").arg(r.totalWeight));
;//zhao
```

ACO

8 Nodes | 16 Nodes | Weight range: 1 - 10 | Regenerate | Run ACO | Reset | Selected: 1 -> 3 | Total path weight: 7



ACO

8 Nodes | 16 Nodes | Weight range: 1 - 10 | Regenerate | Run ACO | Reset | Selected: 0 -> 15 | Total path weight: 8